Performance Tuning: minimizing DBMS response time by identifying and removing bottlenecks so CPU, memory, disk, network, and SQL run efficiently and queries execute fast. **Perspectives: Client/SQL tuning:** Write efficient SQL → correct results, minimal time, minimal server work. **Server/DBMS tuning:** Configure hardware & DBMS (memory, caches, disks, parallelism) for fast response and high throughput. **Query-Processing Bottlenecks: CPU:** overload, poor parallelism, rogue processes. **RAM:** contention, swapping, small buffer cache. **Disk I/O:** slow disks, excessive random reads, insufficient indexing. **Network:** low bandwidth, chatty apps. **SQL/Application:** bad schema, inefficient joins, unnecessary work. **Basic DBMS Architecture:** Client sends SQL → DBMS internal components process: **Listener:** accepts connections. **Optimizer:** creates execution plan. **Scheduler/Executor:** runs plan. **Buffer cache:** stores blocks in RAM. **Data files on disk:** store tables/indexes. **Goal:** maximize cache hits → minimize physical I/O. **Storage & Caches: Data files:** store table/index data. **Extents:** units of file growth. **Tablespaces/Filegroups:** logical data file groups. **Buffer cache:** recently used blocks in RAM. **SQL/Procedure cache:** cached execution plans. **Tuning goal:** maximize cache hits, minimize disk I/O. **Key Server Processes: Listener:** accepts client connections. **Scheduler:** coordinates SQL execution. **Lock manager:** controls locks. **Optimizer:** chooses access paths using stats/rules. **Query Optimization:** Chooses join order, access paths, execution strategy to minimize cost; in distributed DBs, also decides execution location. **Operation Mode: Automatic:** DBMS chooses plan using statistics. **Manual:** Programmer provides hints/forces operations. **Timing: Static:** plan fixed at parse/compile. **Dynamic:** plan chosen at runtime using current data/stats. **Information Use: Cost-Based (CBO):** uses table stats (row counts, distinct values, histograms). **Rule-Based (RBO):** predefined rules (rare). **Query Processing Phases: Parsing:** syntax check, rewrite, optimizer chooses plan, store in SQL cache. **Execution:** run plan (I/O, locks, buffer). **Fetching:** return result rows to client. **Indexes – Role & Types:** Speed up searches, joins, sorting, aggregates. **Trade-offs:** fast reads, extra storage, slower writes. **Types: B-tree:** range scans, ORDER BY. **Hash:** exact lookups. **Bitmap:** few distinct values, complex boolean filters. **Data Sparsity, Selectivity & Index Use: Sparsity:** # of distinct values; high → good index candidate. **Selectivity:** fraction of rows matched; high selectivity → effective index. **Function-based indexes:** index expressions instead of raw columns. **Optimizer Hints:** Optional SQL annotations guiding optimizer. **ALL_ROWS:** minimize total execution time. **FIRST_ROWS(n):** return first n rows quickly. **INDEX(name):** force use of specific index. **Efficient Conditionals:** Filter rows early; prefer column operator literal. Avoid wrapping indexed columns in functions unless indexed. for AND, put the condition most likely to be false first; for OR, put the condition most likely to be true first **DBMS Performance Tuning – Server Side:** Increase buffer cache, tune sort/work areas, parallel execution, partitioning, in-memory DB, faster disks (SSD/NVMe), RAID/SAN. Goal: reduce disk I/O, maximize RAM access. **Query Formulation Checklist:** Decide columns, computations, tables, joins, rows, ordering. Write simple, selective, index-friendly SQL. **SQL-Based Tuning:** Simplify WHERE operands, order AND conditions by selectivity, use/create indexes, leverage optimizer. **Non-SQL-Based Tuning:** Hardware upgrades, caching layers, denormalization, partitioning.**Semantic Caching Analogy:** Reuse results based on meaning, similar to SQL plan cache reuse.**Access Plan:**Optimized sequence of I/O operations from SQL parsing & optimization → “assembly language” version of SQL statements.

Business intelligence (BI) : the tools, technologies, and processes that transform raw operational data into meaningful information for analysis and decision-making. It turns raw data → information → knowledge → wiser decisions. **Benefits:** Better & faster decisions (data-driven). Shared architecture → consistent data models & definitions. Single version of the truth → one centralized data repository. Unified user interface → common reporting/analytics tools. Typical BI Flow: Data sources: internal (OLTP) + external. ETL: Extract → Transform → Load. Storage: Data warehouse + data marts. Access: Reporting, OLAP tools, dashboards. Consumption: Managers, analysts using governed processes. **Operational (OLTP) vs Decision Support (DW/OLAP): Operational data** lives in highly normalized OLTP DBs, focused on fast, accurate transaction processing (current, detailed, many updates). **Decision-support data** is historical, summarized, subject-oriented and multi-dimensional, focused on fast retrieval and complex, broad queries with drill-down (more detail) and roll-up (more aggregation). **Operational DBs vs Data Warehouses** Operational systems are siloed, frequently updated, and optimized for daily operations. Data warehouses are: Integrated: consistent codes, definitions, and formats. Subject-oriented: structured by business subjects (customer, product, region). Time-variant: maintain long-term history. Non-volatile: loaded periodically, rarely updated or deleted. Differences Data currency: operational = current; DW = historical. Granularity: operational = highly detailed; DW = detailed + summarized. Schema: operational = normalized; DW = star/snowflake. Query pattern: operational = transactional; DW = analytical. Volume: DW stores massive historical data. **Star schema** – core warehouse design, widely used in data warehouses; Star schema maps multi-dimensional decision support data into a relational model; easy implementation of multi-dim-data analytics. Fact table: numeric business measures (sales_amt). M:1 relation with dims. Dim tables: are qualifying characteristics that provide additional perspectives to a given fact; dims provide descriptive characteristics through their attributes. Cube view: each fact row represents one point in the multi-dimensional cube (e.g., product × time × location). **Slice and dice** for detailed analytics. PK is a composite key formed by combining foreign keys of all related dims. **SnowFlake:** Dimensional tables can be normalized so that they have their own dim tables - this is done to simplify the design, but requiring navigation across the normalized chains(1:M). Purpose - an attribute hierarchy would provide drill-down/roll-up capability. **When star → snowflake:** When dimension tables are large, hierarchical, or need normalization for integrity/space savings **Operations Summary:** FACT TABLE: Always denormalized, Contains repeated dimension values (date, product, city, etc.), Stores atomic events (sales, clicks, orders). STAR SCHEMA: Dimension tables denormalized, 1 table per dim(city–state–region in same table), Fast queries, simple joins. SNOWFLAKE SCHEMA: Dim tables normalized, Dim split into hierarchy chain (City → State → Region), Saves space, slower queries (more joins) MULTIPLE FACT TABLE COPY: Used in snowflake to avoid long joins Precomputes aggregates for dims. **Data analytics:** Application of math, statistics, machine learning, and modeling to data warehouse content.Explanatory/Descriptive analytics: Understand trends, relationships, patterns. Predictive analytics: Forecast future values/events using models. **OLAP: multi-dim analysis** over a DW to support decision-making and business modeling. Easy to use end user interfaces. Architecture: sources → ETL → DW → OLAP engine/server → client tools (Excel, dashboards, GUIs) → reports, cubes, visualizations. **Data mart**: smaller, single-subject DW subset (e.g., Sales, Marketing). Cheaper, faster to build than a full DW; tailored to 1 grp’s decision-support. **ROLAP**: provides OLAP functionalities to RDBs to store multidim data. optimized query performance for multidim data; good for ad hoc queries and very large databases(VLDBs); more flexible compared to MOLAPS. **MOLAP**: stores data in multidim “cubes” (n-D arrays); often held in memory in cube cache; very fast for predefined dimensions and common cube operations; more rigid (dimensions fixed when cube built). Differences: ROLAP = relational/slow/high flexibility; MOLAP = cubes/fast/low flexibility. **ROLLUP and CUBE** are GROUP BY modifiers - they help generate subtotals for a list of specified columns. Depending on granularity of the columns (eg. US_REGION vs STORE_NUMBER), these subtotals help provide a rolled-up (aggregated) or drilled-down (detailed) analysis of data. Traditional BI uses centralized data warehouses, batch ETL, IT-controlled static reports, and structured data, while modern BI is self-service, real-time, cloud-based, and handles all data types with ML integration. BI and AI both extract insights, but BI explains the past, whereas AI predicts or automates future outcomes. BI supports dashboards, KPIs, and reporting; AI powers forecasting, recommendations, fraud detection, and automation. Data warehouses feed ML models with clean historical data, linking BI’s “what happened” with AI’s “what will happen” for augmented analytics.

Spatial DBs: Spatial DB is optimized to store and query spatial data. Spatial data: has location + extent, **multi-dimensional,** represented by spatial primitives (points, lines, polygons, pixels), **has shape/extent, involves spatial relations (topology/distance), and requires expensive geometry computations.** Geospatial data: spatial data tied to Earth’s geography. Characteristics: Location + size (length/area); spatial autocorrelation (nearby more similar than distant); scale dependence (patterns change with resolution/extent); temporal dependence (change over time) **Entity view:** discrete objects in space (roads, buildings, rivers, etc.); **Field view:** continuous surfaces (elevation, temperature). Spatial eg: CAD (eng drawings/structures); agricultural (precipitation, soil water-holding capacity); 3D data (weather, topological data, satellite images). Geospatial: Disease spread/risk over space/time; drug overdoses/time; census data, income distribution, home prices; store locations (e.g., Starbucks) and (real-time) traffic; crime data; **Spatial Primitives & Topology:** Point: single location; Line/Polyline: connected points;Polygon: closed shape (simple/compound/holes); Raster/Pixel: grid cell with value; Topology: Node: where edges meet; Edge: directed line segment; Face: region enclosed by edges; Encodes: connectivity, adjacency, containment. **Real-world(features) to GIS**: are converted into primitives & each feature also has non-spatial attributes. Vector: discrete objects (points/lines/polygons) Raster: continuous surfaces (elevation,temp). World stored as stacked layers. **Spatial DB Components & Architecture:** A spatial DBMS extends a standard DBMS with:(a)Spatial Data Types (b)Spatial Operations/Functions © Spatial Query Languages: SQL extended with spatial predicates and functions. (d)Spatial Index Structures Core DBMS Supports: Concurrency control, transaction management, Bulk loading of spatial index, Selectivity estimation for spatial queries, Spatial joins and query optimization. **GIS vs. SDBMS:** SDBMS: stores, indexes, queries spatial data efficiently. GIS: performs Earth-related spatial analysis and visualization, often using an SDBMS underneath. **GIS Operations:** Search: attribute, spatial filter; Location analysis: buffer, overlay. Terrain analysis: slope, aspect, watershed. Network/Flow: shortest path, connectivity. Spatial statistics: clustering, autocorrelation. Basic geometric measures: distance, area, direction; **Topological Binary Relations:** disjoint, meet/touch, intersects, within, contains, equals, covers, etc. **Categories of spatial relations:** Topology-based (defined via boundary, interior, exterior); metric-based (distances, angles, usually Euclidean); direction-based (“north of”, etc.); network-based (shortest path). Topological relations can be grouped as proximity, overlap & containment. **Spatial Functions:** Geometry-returning-Buffer(geom, d); Union, Difference, Intersection, SymDiff; ConvexHull; Envelope (MBR); Number-returning-Distance,Area, Length/Perimeter; Analysis Operators-Nearest neighbor, Within distance, Overlay operations (intersection of polygons) **Spatial queries** – types: Spatial measurements (dist btw points,polygon); spatial functions (e.g., find NN); spatial predicates (proximity, containment). **Spatial indexing** – Because spatial data cannot be sorted in 1D, we use **spatial indexes.** B-trees can index spatial data if multidim locations are mapped to sortable 1-D keys (using space-filling curves). Impose an ordering on multi-dim locations where no natural sort order exists. They quantize (x,y) locations into cells of a recursively divided quadtree grid, interleave binary digits of (x,y) to form a 1-D z key in [0,1], then use standard B-tree search on these keys. (Apple's FoundationDB). **R-trees: common spatial idx in practice. Organize data via MBRs (minimum bounding rectangles)**: leaf nodes store MBRs for individual geometries (points, lines, polygons); internal nodes store MBRs covering their children’s MBRs, forming a hierarchy of bounds. Variants: R+-tree, R*-tree, Buddy trees, packed R-trees. k-d tree: Binary tree that recursively splits space along alternating axes; Great for point data. Very good for nearest-neighbor queries. Not ideal for large or disk-based datasets (memory-oriented). K-D-B tree: k-d partitioning (space splitting) + B-tree nodes (disk-friendly, balanced, large fan-out); Good for large datasets on disk. Quadtree (octree in 3D): recursively and adaptively subdivides space into 4 (or 8) children only where needed; nodes are leaves (holding points or empty) or internal (with 4 children); whole structure is a quadtree. **Query Processing: Standard strategy is filter + refine:** filter uses a spatial index (e.g. R-tree over MBRs) to find objects whose MBRs overlap the query window, producing a reduced candidate set; Refine: Run exact geometry checks on candidates to get true results. **Oracle 10g**: SDO_FILTER: Primary filter (fast, MBR only); SDO_RELATE/ SDO_<rel>: Primary + secondary filter (exact); SDO_WITHIN_DISTANCE: Buffer + primary ± secondary filter; SDO_NN: Nearest neighbor; sample: SELECT c.city, c.pop90 FROM proj_cities c WHERE SDO_FILTER(c.location, SDO_GEOMETRY(type,srid, point, SDO_ELEM_INFO_ARRAY, SDO_ORDINATE_ ARRAY)) = 'TRUE'; SDO_TOUCH(a.geom,b.geom) = 'TRUE'; **PostGIS** provides functions such as Distance, Equals, Disjoint,Intersects, Touches, Crosses,Overlaps, Length, Area, Centroid, Within and Contains. **Visualizing spatial data:** Purpose: map non-spatial attributes onto spatial data to reveal patterns, trends, and outliers. Dot maps: 1 dot per event/obs, showing spatial pattern/clustering (incident locations). Proportional symbol maps: symbol size encodes magnitude (mean family income per region). Diagram maps: small charts (bar, pie) at locations/regions for multivariate visualization (racial composition per tract) **Choropleth maps**:shading/color of areas by variable(income,policy impact) **KML point encoding**: `<Point><coordinates>lon,lat[,alt]</coordinates></Point>`

Database Connectivity **Database** Middleware provides an interface between applications and databases, enabling access, query, and management. **Types of Connectivity: Native SQL:** Vendor-specific, optimized for particular DBMS; maintenance burden on programmer. **Microsoft Universal Data Access (UDA):** One interface to access any data source. **ODBC (Open Database Connectivity):** Microsoft’s implementation of SQL Call Level Interface (CLI). Access any relational DB from Windows using SQL. Low-level, requires manual coding; highly portable and widely supported. **Components: ODBC API:** SQLExecDirect, SQLConnect. **Driver Manager:** manages connections, loads drivers. **ODBC Driver:** communicates with DBMS. **DAO + Jet (Data Access Objects + Jet Engine):** Object-oriented API for VB, optimized for MS Access, FoxPro, dBase. Can also access relational DBs. **RDO (Remote Data Objects):** Higher-level API for remote SQL servers, built on ODBC, for client/server apps. **DLLs:** Implement ODBC, DAO, RDO as shared, dynamically linked libraries in Windows. **OLE-DB (Object Linking & Embedding for Databases):** Middleware adding OO functionality to data access. Supports relational & non-relational sources. **Components:** Consumers (applications), Providers (drivers). **ADO (ActiveX Data Objects):** High-level interface built on OLE-DB. Easy, consistent data access; works with scripting languages. **ADO.NET:** Data access in Microsoft .NET framework. Component-based, distributed, heterogeneous applications. **Key Feature: DataSet:** disconnected, in-memory data representation; stored internally as XML, can persist as XML documents. **JDBC (Java Database Connectivity):** Standard Java API for DB access. Pattern: open connection → create statement → execute SQL → iterate ResultSet → close. Benefits: leverages existing tech/skills, supports direct or middleware access, standard SQL, cross-DB access via drivers. **Internet/ Web DB Connectivity:** Enables web/mobile apps to interact with DB via middleware. **3-tier view:** Frontend (UI) → Middleware (web server + app logic) → Backend (DB/data services). Request flow: Browser → web server → dynamic page/script → middleware executes query → format result (HTML/JSON) → browser. Server-side code connects to DB (via JDBC, ADO.NET, ODBC). **Java connectivity:** Applets, sockets, servlets, RMI, CORBA, agents. **Web Application Server:** Interfaces with DBs, search engines, and other services. Supports dynamic pages, query DBs, enforce integrity. IDE support, automated HTML, performance & fault tolerance. Examples: ColdFusion/JRun (CFML-based). **Modern API & Cloud Architectures (MCC Microservices, Containers, Cloud):** Serverless: Cloud model where you run code without managing servers; provider handles infrastructure, scaling, and maintenance, and you pay only for actual usage. **SOAP:** XML, WSDL, strongly typed, enterprise. **REST:** HTTP verbs, JSON, stateless, resource-based. **GraphQL:** client specifies fields, flexible single query. **gRPC:** binary, high-performance RPC for microservices. **WebSocket:** persistent, full-duplex, real-time. **Webhook:** server pushes events asynchronously. **MQTT:** lightweight pub-sub, low bandwidth, IoT. **Cloud Platforms (AWS, Azure, GCP):** Provide compute, containers, storage, networking, managed DBs, analytics, security. **DBaaS:** Managed databases; cloud provider handles provisioning, backups, scaling, patching. Applications connect via JDBC/ODBC as usual.

NOSQL: **Big Data Era : BigUsers:** Millions of users. **BigData:** Huge volumes of data. **BigVariety:** Many new and diverse data types. **BigFlux:** Rapidly changing, fast incoming data. **3V’s of Big Data:** Volume, Variety, Velocity **Data Sources: People:** Clickstreams, purchases, social media, surveys. **People + Devices:** Medical/fitness trackers, GPS, cameras. **Devices:** IoT sensors, scientific instruments. **Why RDBMS Failed for Big Data** Schema rigidity, Difficult horizontal scaling, High latency at large scale, Expensive hardware requirements, OR-mapping overhead, Complex cluster maintenance **NoSQL Databases Advantages over SQL** Flexible schema (schema-less or schema-light) → schema-on-read. High throughput: fast reads/writes. Easy horizontal scaling (sharding, replication). High availability; tolerant to failures. Commodity hardware friendly. Relaxed consistency (BASE principles) **BASE Principle: BA:** Basic Availability – system up most of the time **S:** Soft State – intermediate inconsistencies allowed **E:** Eventual Consistency **JSON & XML in NoSQL** Represent nested data; human-readable; Map naturally to objects (ideal for web APIs); JSON types: number, string, boolean, array, object, null; Handling missing data: omit key, "", or null (semantic difference between null and empty string); Problem with custom formats: difficult to parse, maintain, and extend. **Example JSON** { "name": "Alice", "age": 30, "email": null, "hobbies": ["reading", "cycling"] } ‘Freedom’ from an architectural standpoint **Polyglot Persistence** Use multiple DB types in one application for different tasks; Pick the best DB for each task (e.g., Redis for session data, MongoDB for documents) Pros: Use best tool for each task Cons: Increased complexity, multiple APIs, harder maintenance. **Partitioning/Sharding:** Choose shard key to balance reads/writes **Caching & In-Memory DBs:** Faster access, less backend load, higher throughput **NoSQL Database Families 1. Key-Value Stores** Data: Key → Value (entire record is value); Schema-less, key-based queries only; Fast lookups; full-value updates Use cases: caching, session stores, high-throughput lookups **Examples: Redis:** Stores strings, lists, sets, hashes, sorted sets, streams **Memcached:** Temporary cache **Dynamo:** MD5 hashed key, replicated on 3 nodes, values as BLOBs **Advantages:** Faster access, lower backend load High throughput, cloud-based horizontal scale-out **2. Column-Family Databases** Store: Row Key → Columns (rows need not match) Vertical fragmentation → faster queries by retrieving only relevant columns Supercolumns: Grouped columns; Supercolumn Families: row → supercolumn → columns; Ideal for wide-column analytics, very large distributed systems **Examples:** Cassandra, HBase, BigTable, RedShift **3. Document Databases** Store data as documents (JSON, BSON, XML) Key → Document (value), supports nested objects and arrays Queries: key-value, range, geo-spatial, text (full-text search), aggregation (count, min, max, avg) Better than key-value DBs for complex queries **Examples:** MongoDB, CouchDB **4. Graph Databases (most flexible)** Data: Nodes + Edges + Properties; Ideal for highly connected data: social networks, recommendations, knowledge graphs Allows fast relationship traversal **Examples:** Neo4j **Triple/Quad Stores (RDF Resource Description Framework)** Store data as **triples**: (subject, predicate (relationship), object) A Knowledge Graph is constructed by connecting these triples, where the object of one triple can serve as the subject of another Quad stores add a 4th element (context) Useful for: knowledge graphs, AI reasoning systems Storage modes: **In-memory:** fastest, volatile **Native:** vendor’s storage engine **Non-native:** uses another DB (e.g., MySQL) **Vector Databases** Store high-dimensional embeddings Use approximate nearest neighbor indexes for semantic search, recommendations, **LLM RAG** pipelines Similarity-based retrieval (cosine, Euclidean, dot-product) **ARFF (WEKA) Example** @RELATION coffee_pref @ATTRIBUTE age NUMERIC @ATTRIBUTE likes_coffee {yes, no} @DATA 25, yes 40, no Examples of **large traditional data**: logs, transactions, sensor streams. **DNA** as ultra-dense data storage. To empower managers/domain experts to analyze Big Data without engineers, build or adopt a **turnkey data science platform**, offering: Point-and-click data ingestion from DBs and streams Auto-cleaning & transformation Built-in ML & visualization Dashboard/report builders Cloud back-end and scalability Examples: **Qubole, Splunk, Azure Databricks, Netezza, Dataiku**.

MapReduce : Motivation Modern databases store data as key/value pairs, resulting in explosive growth when it comes to number of rows and file sizes. Traditional, sequential, single machine oriented access does NOT work at all - what is needed is a parallel way to process data (SIMD). **MapReduce core idea**: **Input Split:** Input data is divided into blocks (HDFS/Hadoop: 64–256MB) and distributed across nodes. **Map Phase:** Runs in parallel on each block. Transforms input (k1, v1) pairs → emits intermediate (k2, v2) pairs. **Combiner (Optional):** Runs locally on mapper nodes before shuffle. Aggregates intermediate results to reduce network traffic and load on reducers. Safe only for associative & commutative operations (e.g., sum, min, max). Use-cases: partial sums, local aggregations. **Shuffle and Sort:** System groups all intermediate (k2, v2) by key: (k2, [v2]). Network-heavy; minimized by combiners. **Reduce Phase** Processes grouped keys to produce final results (k2, v3). Rule: Reducers wait for mappers to complete (unless streaming is used). **Output:** Final aggregated results written to distributed storage (HDFS, GFS).; **Diagram:** Input Splits -> Map Tasks (k1,v1 → k2,v2)->[Combiner optional] Local aggregation at mapper -> Shuffle & Sort Group by key (k2) -> Reduce Tasks (k2,[v2] → k2,v3) ->Output(HDFS/GFS/HBase) **High level**: User-defined logic (Map, Reduce, Combiner, Partitioner, Hive/Pig/Spark pipelines, logical flow: Split → Map → Shuffle → Reduce). **Low level:** System/runtime internals (InputFormat/RecordReader, task scheduling, spill/merge, shuffle mechanics, sort/group, JVM task containers, HDFS blocks, NameNode/DataNodes).**Formal MapReduce model** Input is a multiset of (k1, v1) pairs. The Map function map(k1, v1) emits zero or more (k2, v2) pairs. The system groups all intermediate pairs by k2 into (k2, [v2]). The Reduce function reduce(k2, [v2]) emits zero or more final values v3 (often aggregated numbers or summaries). **Word Count example** Goal: count occurrences of each word in a huge text corpus. Map: for each word w in an input line, emit (w, 1). Shuffle/sort: system groups all counts by word → (w, [1, 1, …]). Reduce: sum each list to get (w, total_count). In Hadoop, this can be implemented in **Java** (Mapper + Reducer classes) or via streaming (e.g., **Python** scripts reading/writing “word count” lines); Hadoop guarantees mapper outputs are sorted by key before reducers see them. **Distributed storage: GFS and HDFS** **Google File System (GFS)** is a distributed file system designed to store very large files across many machines for MapReduce; it splits files into large chunks stored on chunkservers, with a master tracking metadata and locations. **Hadoop Distributed File System (HDFS)** is the open-source analog used in Hadoop clusters, with a single NameNode managing metadata and multiple DataNodes storing replicated blocks; it provides reliable, high-throughput storage for MapReduce and related frameworks. **Hadoop and its ecosystem** Hadoop is the open-source implementation of GFS- and MapReduce-style ideas, originally emerging from the Nutch search engine project. Core components are HDFS (storage) and MapReduce (batch processing). On top of Hadoop there are higher-level tools: **Pig** (data-flow scripts compiled to MR), **Hive** (SQL-like warehouse on Hadoop), coordination and workflow tools (**Zookeeper**, **Oozie**), data integration tools (**Sqoop**, **Flume**), ML and analytics libraries (e.g., **Mahout**), management/monitoring tools (such as **Ambari**), and NoSQL systems like **HBase** (column-family store on HDFS). Typical pattern: import data into HDFS, process via MapReduce/Pig/Hive, then store results back to HDFS, HBase, or external warehouses for reports and models. **YARN and MapReduce v2** In classic MRv1, a single JobTracker handled job scheduling and resource management, with TaskTrackers on workers; this centralized design was a bottleneck and MR-only. YARN (Yet Another Resource Negotiator) separates resource management from application logic: a ResourceManager globally schedules resources, NodeManagers manage containers on each node, and an ApplicationMaster coordinates each application’s tasks. **Advantages over MRv1** YARN allows multiple engines (MapReduce, Spark, graph engines, streaming, etc.) to share the same cluster, improving scalability and supporting more than just batch jobs. **HIVE, PIG, MUSKETEER: Hive** provides a SQL-like language called HQL that lets users run analytical queries on large datasets stored in Hadoop without writing MapReduce code. Hive translates HQL queries into MapReducejobs, giving the scalability of Hadoop with a familiar SQL interface. **Pig**, on the other hand, is a dataflow system that uses Pig Latin, a scripting language for building ETL-style pipelines. Pig Latin scripts describe a series of operations (like join, filter, group), which are automatically compiled into MapReduce jobs. Hive is more declarative and table-oriented, while Pig is more procedural and suitable for complex data-processing workflows.. **Musketeer** is an experimental framework that decouples front-end langs like Hive or Pig from specific back-end engines. It allows the same workflow to run on different systems and even lets different workflow components execute on different engines, improving portability and efficiency. **Cloud Infrastructure and VMs** Cloud platforms like AWS EC2/S3, Microsoft Azure, and Google Cloud provide elastic scaling, high availability, and fast access, making them ideal for deploying Big Data systems. Hadoop clusters can be easily set up on these cloud environments, avoiding heavy on-prem setup. A **virtual machine (VM)** simulates a full computer inside a host system, allowing users to run isolated environments for tools and distributed frameworks. Hadoop can also be experimented with using VM-based sandboxes such as HortonWorks Sandbox, MapR Sandbox, Cloudera CDH VM, Oracle Big Data Lite VM, IBM BigInsights VM, and Talend Big Data Sandbox. For real-time workloads, solutions like Blaze VM support *streaming analytics*, which continuously process sensor, app, and event data to detect patterns and produce insights in real time. These VMs provide a simple, self-contained environment to learn, develop, and test Big Data and streaming applications without complex manual setup. **Beyond MapReduce: Spark and friends** Apache Spark is an in-memory distributed processing engine that can run on HDFS/YARN and other cluster managers. It generalizes the MapReduce model to a DAG of operations and keeps data in memory between steps using RDDs (Resilient Distributed Datasets), making iterative algorithms (ML, graph processing) and interactive queries much faster than repeated MR jobs. Spark provides a modular ecosystem: Spark Core (RDD engine), Spark Streaming (stream processing), SparkSQL(DataFrames & SQL), MLlib (), GraphX (graph analytics). **Apache Flink** is a distributed engine for high-throughput, low-latency processing of unbounded and bounded data streams. **Bulk Synchronous Parallel (BSP)** The model is an **alternative to MapReduce** designed for iterative graph-based computations. BSP runs on multiple processors that operate independently in a series of **supersteps**, each consisting of **(1) local computation** on each processor’s own data, **(2) communication** where processors exchange messages, and **(3) a barrier synchronization** where all processors wait before the next superstep. Google’s **Pregel** implements BSP with a “think like a vertex” model for large-scale graph processing; open-source equivalents include **Giraph**, **Hama**, GoldenOrb, and GPS. **Hama** is a general BSP engine capable of graphs, ML, scientific computing, matrix algorithms, etc., while **Giraph** focuses only on graph workloads. BSP is particularly suitable for massive iterative graph tasks such as PageRank, label propagation, and friend-of-friend scoring—Facebook processed **one trillion edges** using Giraph. Overall, BSP provides a powerful alternative to MR by enabling fast in-memory iteration and efficient message passing across vertices.

DataMining: The science of extracting **useful patterns, models, and insights** from (often large) observational datasets. It discovers **previously unknown relationships**, summarizes data, and supports **prediction** and **decision-making**. Data mining is essentially **statistics at scale**: broader in scope, more automated, more algorithmic, and designed for **large, complex, high-dimensional data** rather than traditional small-sample inference. **Data mining vs machine learning** Data mining = whole process: ask question, explore data, prepare, model, deploy, act, evaluate, repeat. Machine learning = core toolbox inside DM: train models on existing data to learn patterns, then apply models to new data. **DM & related fields**: Data mining sits between databases (storing/querying big data), statistics (inference and uncertainty), AI (search, heuristics), and pattern recognition/ML (classification, clustering, prediction). **Data mining cycle**: Ask a business/research question; use available data to Explore → Prepare → Model; then Implement → Act → Evaluate in production; new behavior generates new data and new questions → cycle repeats. Data Science based analysis (Data Science Life Cycle): Data Mining, Machine Learning, Regression, Classification **High-level vs. Low-level Pipelines High level**: Pipelines are DAGs with nodes (processing steps) and edges (dependencies), allowing parallel execution. **Low level**: Frameworks like **MapReduce** implement this via massively parallel **Map → (shuffle/sort) → Reduce**, e.g., WordCount. **3 Stages of a Data Pipeline Data ingestion** – collect from many sources/nodes **Data processing** – clean/transform using multiple parallel nodes **Data output** – deliver final data to storage, dashboards, or ML models. **Typical uses of data mining** Predict who buys what and when; set insurance/credit risk; predict equipment failures and maintenance needs; detect churn and fraud; manage financial risk; improve marketing response and ROI; match offers to customers. **Generic application patterns** Profiling/segmentation: group customers by behavior/needs. Cross-sell/up-sell: predict likely next purchases. Acquisition/retention: find high-value prospects and at-risk customers. Campaign management: right offer to right person at right time. Lifetime value: focus on customers with highest future margin/retention. **Learning paradigms**: Supervised learning: manually (software) labeled outcomes. Unsupervised learning: no labels, find structure. Semi-supervised: mix of labeled + unlabeled. Self-supervised: labels derived automatically from the data itself. **Main categories of data mining algorithms** **Classification** = labeling data into discrete classes. **Decision trees** Tree model that recursively splits data using feature tests; each internal node is a condition, each leaf stores a class or numeric prediction; classification trees predict categories, regression trees predict numbers; very interpretable “if–then” rule set. e.g., loan approval rules. **Support Vector Machines (SVM)**: Supervised classifiers that find a separating hyperplane between classes with maximum margin (distance to closest points); support vectors are boundary points that define the separator; extended with kernels and soft margins to handle non-linear, non-separable data. e.g., spam vs. non-spam email. **k-Nearest Neighbors (kNN)**: Lazy supervised method: to classify a new point, look at its k closest labeled neighbors in feature space (after normalizing features) and use majority vote; no explicit training model, all work is at query time. e.g., image recognition.

Naive Bayes Probabilistic classifier that uses Bayes' rule with a simplifying assumption that features are conditionally independent given the class; estimate class priors and feature likelihoods from data; for a new sample, compute class scores and pick the class with maximum posterior. e.g., text classification. **Logistic regression** Logistic regression is a binary classification algorithm that first fits a linear regression model to separate two classes, producing a value that can range from −∞ to +∞. This value is then passed through the logistic (sigmoid) function, which converts it into a probability between 0 and 1. If the probability ≥ 0.5, the model predicts Class 1; otherwise, Class 0. Logistic regression basically converts a continuous regression output into a meaningful binary decision. e.g., churn prediction. **Main categories of data mining algorithms** **Classification** = labeling data into discrete classes. **Decision trees** Tree model that recursively splits data using feature tests; each internal node is a condition, each leaf stores a class or numeric prediction; classification trees predict categories, regression trees predict numbers; very interpretable "if–then" rule set. e.g., loan approval rules. **Support Vector Machines (SVM)**: Supervised classifiers that find a separating hyperplane between classes with maximum margin (distance to closest points); support vectors are boundary points that define the separator; extended with kernels and soft margins to handle non-linear, non-separable data. e.g., spam vs. non-spam email. **k-Nearest Neighbors (kNN)**: Lazy supervised method: to classify a new point, look at its k closest labeled neighbors in feature space (after normalizing features) and use majority vote; no explicit training model, all work is at query time. e.g., image recognition. **Naive Bayes** Probabilistic classifier that uses Bayes' rule with a simplifying assumption that features are conditionally independent given the class; estimate class priors and feature likelihoods from data; for a new sample, compute class scores and pick the class with maximum posterior. e.g., text classification. **Logistic regression** Logistic regression is a binary classification algorithm that first fits a linear regression model to separate two classes, producing a value that can range from −∞ to +∞. This value is then passed through the logistic (sigmoid) function, which converts it into a probability between 0 and 1. If the probability ≥ 0.5, the model predicts Class 1; otherwise, Class 0. Logistic regression basically converts a continuous regression output into a meaningful binary decision. e.g., churn prediction. **Clustering (unsupervised)** = Discovers natural groups. grouping similar instances with no labels. **K-means** Unsupervised method that partitions data into k clusters by iteratively assigning points to the nearest centroid and recomputing centroids as means; used for segmentation; choose k by looking at SSE vs k and finding an "elbow". e.g., customer segmentation. **Hierarchical (clusters of clusters of..).** Builds nested clusters (tree of clusters, dendrogram). e.g., customer segmentation. **Agglomerative:** start with individual points and merge closest clusters. **Divisive:** start with one big cluster and split. Good for exploring multi-level structure. How do we decide what to merge? A popular strategy - pick clusters that lead to the smallest increase in sum of squared distances. **Expectation-Maximization (EM)** The EM algorithm estimates model parameters when some data is hidden. It tackles the loop where parameters depend on unknown latent variables, and those variables depend on the parameters. Starting with initial guesses, EM alternates between the E-step (estimating the latent variable distribution) and the M-step (updating parameters to maximize expected likelihood) until convergence. It's simple, effective, and widely used—for example, in Gaussian Mixture Models for soft clustering. e.g., soft customer grouping. **Regression** = predicting numeric outcomes (and sometimes spotting outliers). **Linear regression** Models numeric outcome as linear function of one or more predictors plus noise; in 2D, fits best line y=mx+c; in multiple dimensions, fits hyperplane; summarizes linear relationships & trends. e.g., house price prediction. **Non-linear & non-parametric regression** (alternatives for predicting numerical targets) Non-linear regression fits curved relationships (e.g., polynomials, more complex functions). Non-parametric methods make fewer assumptions and let the data shape flexible surfaces, at the cost of needing more data and risking overfitting. e.g., curved trend forecasting. **Regression Trees (non-parametric, rule-based)** – e.g., predicting equipment lifetime. **Association** = finding items/events that co-occur together. **Apriori / association rules** Association analysis finds itemsets that frequently occur together in transaction data and rules A ⇒ B with high support (how often A and B appear together) and confidence (how often B appears given A). Used for market-basket analysis, recommendations, and promotions. **Classification vs clustering** Classification uses known labels to assign new data to predefined classes. Clustering has no labels and discovers groups based on similarity; labels may be added later by humans. **Data terminology & geometric view** Rows = samples/data points; columns = attributes/features; optional label column = target/output. Each sample is a point in multi-dimensional feature space; data mining finds patterns, clusters, and decision boundaries in this cloud of points. **Ensemble learning** Meta-approach that combines predictions from many models to improve accuracy and stability; often reduces variance and error vs single models. **Boosting** It's an ensemble learning technique that combines many **weak learners** to form one strong, highly accurate model. It works in **sequential steps**: each new model is trained to focus more on the mistakes made by the previous models. Incorrectly classified data points are given higher weight. All weak learners are combined(through a weighted vote) to produce a strong classifier. e.g., robust predictions on noisy data. **Bagging (Bootstrap Aggregating)** Trains multiple models on different bootstrap samples of the data and averages/votes their predictions; reduces variance and improves robustness. e.g., improving weak models for fraud detection. **Random Forest** Bagging of many decision trees with extra randomness: each tree sees a bootstrap sample and, at each split, only a random subset of features; final prediction is majority vote (classification) or average (regression); strong, stable, widely used tree ensemble. e.g., general-purpose classification/regression.

Machine Learning: **Data-Driven ML (Connectionist)** Intelligence is learned from data through a structured pipeline. Data-driven AI (a subset of ML) relies on statistics, linear algebra, and calculus, using symbolic, reinforcement, and neural approaches to learn from data (supervised, unsupervised, semi-supervised) to optimize performance and business outcomes. **Data-Driven ML Steps for Prediction or Decision-Making: Data Collection:** gather representative datasets. **Data Preprocessing:** clean data, handle missing values. **Feature Engineering:** select or create relevant features. **Model Training:** use frameworks like PyTorch or TensorFlow. **Model Evaluation:** metrics such as accuracy, precision, recall, F1-score. Measure model performance on validation/test data. Identify overfitting or underfitting. Guide model improvement and selection. **Connectionist / Neural Workflow:** Collect (and optionally label) data, iteratively train a network by adjusting parameters to reduce loss, then deploy it for predictions—loosely inspired by biological neurons. **History & Applications:** Modern deep learning uses large datasets to find patterns and generate new data. Key applications: computer vision, self-driving cars, chatbots, and Generative AI. Recovery from AI winters was driven by better algorithms, more data, specialized hardware, and practical use cases. **Supervised, unsupervised, semi-supervised learning** Supervised learning maps labeled inputs to outputs, unsupervised learning discovers hidden structure/manifolds in unlabeled data, and semi-supervised learning combines a small labeled set with lots of unlabeled data so structure plus labels guide the model. **Reinforcement learning (RL)** RL models an agent interacting with an environment to learn a policy that maximizes long-term reward, often via MDPs/POMDPs and hierarchical decompositions, and is used for robotics, navigation, and sequential decisions. **Artificial neural networks (ANNs)** ANNs are networks of interconnected, weighted nonlinear units that are trained on labeled examples to learn patterns and then use those patterns to make predictions on new data. **Single neuron – computation** A single neuron computes a weighted sum of its inputs plus a bias and passes it through a nonlinear activation (e.g., sigmoid), producing an output used by downstream neurons or as a prediction. **NN applications & brain analogy** Neural nets can learn patterns in almost any structured data (classification, anomaly detection, forecasting, etc.) and are loosely analogous to the brain's huge network of neurons and adaptive synaptic weights. **Artificial neurons & activation functions** (Most Important Part of AI) Different activation functions (linear, threshold, ReLU, sigmoid, etc.) map net input to output and provide the nonlinearity and gradients needed for neural nets to learn complex patterns via backprop. Enable universal approximation Without them the network would behave like one big linear function. **Sigmoid Function** keep networks from collapsing into a single linear mapping, and a typical ML pipeline goes from understanding the business problem through data prep and training to deployment and ongoing monitoring. Formula: σ(x) = 1 / (1 + e^-x) Used in neural networks and logistic regression. Thresholding: p > 0.5 → class 1, else class 0. **NN theory & layered architectures** structure of layers and connections in a neural network. Neural nets can be seen as directed graphs or large dynamic systems where layers of units transform inputs into higher-level features and learning means adjusting parameters in a model y = f(x; W) to minimize a loss. **Backpropagation learning** Backprop iteratively adjusts weights by propagating gradients from the output layer backwards, using hyperparameters like learning rate and momentum to reduce loss and assign "credit/blame" to individual units. **Hierarchical features, universal approximation & simple examples** Multi-layer nets learn hierarchical features (e.g., pixels → parts → faces) and, with enough neurons and the right nonlinearities, can approximate any continuous function, as seen in simple classifiers and regression examples. **XOR, Softmax & why deep NNs matter** Solving non-linearly separable problems like XOR requires multi-layer networks, Softmax extends sigmoid to multi-class output, and deep NNs took off thanks to big data, better algorithms, and powerful hardware. **Neuron summary & implementation** A simulated neuron corresponds to biological dendrites, soma, and axon: it sums weighted inputs, passes them through an activation, and emits an output signal in code. **Supervised ML as giant nonlinear function & deep learning** Supervised ML can be viewed as learning a big nonlinear function y = F(x) by tuning architecture and hyperparameters so predictions match targets, with deep learning using many layered nonlinear units enabled by modern data and compute. **Deep learning uses, "deepness" & architectures (incl. RNNs)** Deep nets with many hidden layers power state-of-the-art image, speech, and NLP systems using architectures like CNNs, RNNs/LSTMs, TCNs, HTM, and Transformers, where "depth" is the number of intermediate layers.

Convolution & CNNs Convolution slides small kernels over inputs to extract features, and CNNs stack many such layers (with ReLU and pooling) to learn complex patterns in images, speech, and other signals, enabling systems like YOLO for real-time detection. **CNN Misclassification Cause** Insufficient or no additional data for proper generalization. **GANs, EBMs, VAEs & encoder–decoder pairs** Generative models like GANs (adversarial generator–discriminator), EBMs, and encoder–decoder VAEs learn data distributions and latent spaces so they can generate new realistic samples from noise or latent codes. **Autoencoders (AE)**: Encode an input into a single point in latent space. Decoder tries to reconstruct the exact input. Latent space can be messy and not smooth. Good for compression, not great for generating new data. **Variational Autoencoders**: Encode input into a distribution (mean + variance), not a single point. Sample from this distribution before decoding. Latent space becomes smooth, continuous, so nearby points produce similar outputs. Can generate new, realistic samples, not just reconstruct. **Loss Functions in ML** measure how far a model's predictions deviate from the true values, providing a numerical score of error. This score is then minimized during training to adjust the model's parameters and improve accuracy. **Transfer Learning from one app to another** Save weights and/or architecture file for model reuse. Practical applications: self-driving cars, smartphone apps for identifying objects (birds, flowers, clouds, mushrooms). **Source of Bias in ML** Chiefly from dataset (imbalanced labels, incomplete representation). Algorithmic sources also exist (loss function calculation). **Bias Fix** Audit data labels, resample or augment to balance classes. **Tools for ML/DM** TensorBoard: visualize TensorFlow graphs. Model packaging: CoreML, CreateML, Turi, ONNX. High-level APIs: Keras, PyTorch (also mxnet, scikit-learn). Code-free data processing: WEKA, KNIME, RapidMiner, Orange. Hardware-accelerated solutions (beyond GPU/TPU): Coral, Jetson Nano, Movidius NCS. **Applications & demos – sampler** ML is now deployed across thousands of real-world applications—from vision, speech, medicine, and security to games and creative tools—demonstrating mature, production-ready capabilities. **Problems, hype & critical views** Key concerns include biased data, deepfakes, adversarial weakness, lack of explainability, and concentrated power, leading many researchers to critique current ML/AI limits and hype. Fundamental issues like poor data quality, biased datasets, or intrinsic problem ambiguity cannot be solved by faster processing, more memory, or more training data. **Current directions & major players** Cutting-edge work spans transformers, neuromorphic and optical computing, GNNs, contrastive learning, pruning, and explainability, driven by major tech companies and cloud providers worldwide. Weights: parameters that determine the influence of inputs on outputs.

DATA VIZ: Data visualization transforms raw data into graphical formats, allowing patterns to be recognized, insights to be communicated, and informed decisions to be made. It is applicable across any kind of dataset and domain, turning complex numbers into intuitive visual stories.. **Single-Variable (1-D) Visualization** Charts that display one variable are useful for understanding distributions or relative importance. Good design dramatically improves clarity. Common examples include: **Pie charts / Donut charts** – show proportion of categories. *Example*: Donut chart for popularity of game consoles (Switch, PlayStation, Xbox, Wii). **Histograms** – show frequency distributions of continuous variables. *Example*: Histogram for popularity of various coding languages in 2020. **Bar charts** – compare quantities across categories. **Bubble charts** – encode an extra variable using bubble size. *Example*: Bubble plot for popularity of social media platforms. **Word clouds** – highlight relative importance of textual terms. **Multivariate Data Visualization** Multivariate graphics encode multiple variables simultaneously, often in a single 2-D visualization. **Minard's Napoleon chart** is a classical example, showing army size, location, and temperature on a single map. **Modern potential**: Today, Minard could use animation, interactivity, and overlays (like weather or satellite data) to give a richer story, while Florence Nightingale's coxcomb diagrams could similarly exploit dynamic dashboards and interactivity to emphasize public health insights more effectively.**Spatial Data Visualization** Mapping data on geography reveals spatial patterns and supports planning. Common methods: **Points** – location-specific data. **Choropleths** – intensity-based coloring of regions. e.g., hours spent online per US state/county. **Heatmaps** – show density or concentration patterns. Applications: demographics, public health, retail layout. **Spatio-Temporal Data** Overlaying time on spatial maps can reveal evolution and uncertainty. Examples include hurricane tracks, wildfire spread, and epidemic maps. These often employ: **Animations** or **time-sequenced maps** to show changes dynamically. **Interactivity and Animation** Filters, drill-downs, and time sliders let users explore and discover hidden patterns. Enhances understanding of trends and correlations over time. **Real-Time Visualization** Dashboards for live data, e.g., traffic, stocks, earthquakes, cyberattacks. Provides immediacy and supports real-time decision-making. **Network Visualization** Nodes and edges, with visual encodings (size, color, thickness, direction), reveal relationships in connected systems. Examples: social networks, collaboration maps, or COVID-19 contact tracing. **Advanced and Immersive Modes of Data Presentation** To boost engagement and utility, we can extend beyond traditional screens:**Virtual Reality (VR)** – Users are visually immersed in the data, even overlaying it on 3D scenes. **Augmented Reality (AR)** – Data visualized on real-world surfaces for collaborative analysis (tabletops, walls). **Holograms** – 3D visualizations for physical phenomena, e.g., polar ice caps melting. **3D Printing** – Physical models of data, optionally in color. **Projection Mapping** – Data projected onto real-world surfaces for public or interactive display **Notable Visualization Examples** Collaboration maps, language maps, shot charts, media bias plots, and daily routine timelines illustrate intuitive storytelling across diverse topics. **Data Visualization Tools Programming libraries**: R, Python (matplotlib, seaborn, Plotly). **Web/JS**: D3.js, Plotly.js. **Business Intelligence**: Tableau, Qlik. **Online chart builders**: Flourish, Datawrapper. **The "Science" of Data Visualization** Combining art and science, data visualization relies on: **Perception, color theory, design principles Semiotics** (meaning of symbols) **Graphic variables** (position, size, shape, color) **Matching data types** to visualization forms: tables, charts, maps, networks Goal: highlight insights rather than technology. **Good Design and Resources** Focus on clarity, insight, and accessibility. Resources: Edward Tufte books, online galleries, talks, and newsletters. **Today's Business Intelligence (BI): Quantum Leap** Modern BI surpasses classical methods in three major ways: **Interactivity and Animation** – Dynamic dashboards, real-time exploration. **Real-Time Analytics** – Continuous data streams enable immediate insights. **Cloud & Mobile Processing** – Scalable, collaborative, and accessible anywhere.

Data Governance Data work isn't just storage/analytics; you must also manage privacy, security, ethics, trust, compliance, and governance as data pipelines grow. **Privacy** Fair Information Practices efforts in organizations followed five tenants: Openness, Disclosure, Secondary usage limits, Correctability, Security. Today's massive data collection (tracking, medical records, location, search, collaborative filtering, facial recognition) mean our lives are highly observable and the idea of "privacy" may need redefining. **Security** Security focuses on blocking unauthorized access (breaches, IoT attacks) through technical controls like encryption and access restrictions, while Data privacy on the other hand, has to do with (limiting) AUTHORIZED access to data. **Ethics** Ethical issues include biased AI (e.g., unfair face recognition), autonomous weapons, deepfakes and information warfare, and require bias reduction, interpretability, and responsible ML use. **Trust** People's trust in organizations handling their data depends on transparency, clear user benefits, and taking responsibility when things go wrong. **Compliance** Compliance is a LEGAL/REGULATORY issue - what laws be passed, to help citizens have/gain control over their data. Laws like the EU's GDPR(General Data Protection Regulation) (strong, unified, "right to be forgotten") contrast with the U.S's fragmented, changing privacy rules with administration, shaping how organizations must handle personal data. **Governance (overview)** Data governance sets the rules, policies, and processes for how data is collected, stored, managed, and used to maintain high quality throughout its lifecycle. Data Governance (DG) is defined as the exercise of authority and control (planning, monitoring, and enforcement) over the management of data assets. In other words, Governance is about POLICIES, which can be seen as complementary to Curation. **Data curation** Curation uses processes and technologies(methods and tools) to turn raw data into high-quality, reusable, interoperable data, with governance providing the policies and curation doing the practical work. **Data provenance** Data provenance documents the inputs, entities, systems, and processes that influence data of interest, in effect providing a historical record of the data and its origins. Provenance has to do with origins, while lineage has to do with tracing data's 'journey' to the current point of usage. Provenance/lineage is a form of metadata that needs to be added to data, during curation. Provenance (lineage) records where data came from and how it was transformed, building trust and enabling correct, safe use—especially in scientific and safety-critical settings. **MDM – Master Data Management**: It is the practice of managing an organization's "master data"—the core, critical information that is shared across business processes and systems. The goal of MDM is to maintain a single, accurate, complete, and timely reference for this data, ensuring consistency across the organization. Without MDM, duplicating or replicating the same data across multiple systems can lead to inconsistencies, errors, wasted storage, and increased network traffic. By integrating data from various sources, MDM helps organizations make better decisions, maintain data quality, and streamline operations. **Governance, Security, Privacy – "trinity"** Weak governance often leads to costly security/privacy breaches, so organizations should invest in strong policies, culture, and technical protections to manage data risk. **Deepfakes**: AI-generated realistic media (videos, images, audio) that can be used to misinform, manipulate public opinion, or impersonate individuals. **Information warfare**: Coordinated campaigns using data and media to influence politics, economics, or social behavior.

GEN AI : **Classic Machine Learning vs Generative AI Classic ML**: Learns mappings from inputs to outputs (e.g., classification, regression). Relies heavily on labeled data. **Generative AI**: Learns the **underlying data distribution** so it can create **new, realistic samples** rather than just predicting outputs. **Generative AI Revolution – GANs Generative Adversarial Networks (GANs)**: Consist of a **generator** and a **discriminator** in adversarial training. The generator creates synthetic examples from noise, while the discriminator evaluates realism. Through this feedback loop, the generator learns to produce highly realistic data (e.g., human faces, art). **Neural Style Transfer** Uses a **pre-trained CNN** and optimizes pixel values to **blend the content of one image with the style of another**. Balances **content loss** (preserving the original image) and **style loss** (adopting the new style). **Encoders, Decoders, Autoencoders, VAEs Encoders** compress data into a **latent space**; **decoders** reconstruct it. **Autoencoders**: Learn to recreate the input exactly. **Variational Autoencoders (VAEs)**: Learn a **latent distribution**, allowing meaningful **sampling and interpolation** to generate new data. **Transformers** Use **self-attention** to model relationships between tokens. Power models like **BERT** and **GPT**. Applications extend beyond text to **vision, time series, music, and more**. **Fine-Tuning LLMs (FT) Fine-tuning** adapts a general LLM to specific domains. **Parameter-Efficient Fine-Tuning (PEFT)**: Techniques like **ReFT** or **LoReFT** tweak only a small subset of parameters to achieve high performance at lower cost than classic FT. **RAG – Retrieval-Augmented Generation** Combines **LLMs with external knowledge retrieval**. Process: Retrieve documents → feed along with query → generate enhanced answer. Effectiveness depends heavily on **retrieval quality and document chunking**. **GenAI for Non-Plaintext Content** Modern generative methods (e.g., **diffusion models**) can synthesize: **Images**, **video**, **audio**, **3D scenes**, **molecules**, **designs** Not limited to text. **New Directions in LLMs & GenAI** Emerging architectures: **SLMs**, **MoE**, **world models** Improvements in **hardware, model compression, RAG/agent tooling** Standards: **MCP**, **DSPy** Trend: More **efficient, agentic, and specialized systems. The OPL Stack O**: OpenAI (or other LLM/embedding provider) **P**: Pinecone (vector database for embedding storage and similarity search) **L**: LangChain (framework for orchestration and RAG pipelines) Purpose: Build **RAG applications**, connecting LLMs to external databases for domain-specific knowledge. **Up-and-Coming Query Languages NLP-based querying**: Query systems using natural language instead of formal syntax. **Graph Query Language (GQL)**: Query **knowledge graphs** efficiently, capturing relationships between concepts. **AGI (Artificial General Intelligence)** can learn and apply knowledge across diverse tasks like humans. It learns concepts from real-world interactions—physical phenomena, sensations, and social behavior—building a structured, interconnected understanding. Unlike traditional deep learning, it doesn't rely heavily on massive datasets.